Go-ascended

Ton outil go-reforged est robuste, rapide et testé. Mais chacune de ses règles est écrite en dur dans le code : pour ajouter un modificateur, il faut recompiler. Dans ce projet, ton outil devient un véritable moteur. Les règles seront décrites dans un petit langage que tu vas concevoir, analyser, vérifier et exécuter. Ton moteur devra aussi parler à un éditeur de code, résister à des fichiers malveillants et traiter plusieurs gigaoctets de texte.

Objectifs

Dans ce projet, tu vas :

- écrire un **langage de règles** complet : analyse lexicale, analyse syntaxique, vérification des types et exécution ;
- réécrire **toutes** les règles de go-reforged dans ce langage ;
- produire des **diagnostics de qualité compilateur** et une **carte des sources** (source map) ;
- implémenter un **serveur de langage** (LSP) qu'un éditeur comme VS Code ou Neovim peut utiliser ;
- traiter les fichiers et les règles comme des **entrées hostiles**, avec un modèle de menaces écrit ;
- garantir des **performances** mesurées et un résultat **déterministe**.

Le projet sera corrigé par des auditeurs, qui seront d'autres étudiants. Tu seras toi aussi auditeur. Une partie de l'audit consiste à **attaquer** ton programme (partie 6) : prépare-toi en l'attaquant toi-même.

Prérequis : avoir validé go-reforged. Go 1.24 ou supérieur (nécessaire pour `os.Root`).

Introduction

- Ton projet doit être écrit en **Go**.
- Le code doit respecter les **bonnes pratiques**.
- Seuls les packages de la bibliothèque standard de Go sont autorisés.
- Les packages `unsafe`, `reflect`, `plugin` et `os/exec` sont **interdits**.
- Le programme ne doit **jamais** paniquer, quelle que soit l'entrée : texte, fichier de règles ou message LSP.
- `go vet ./...` ne doit signaler aucun problème.
- `go test -race ./...` doit passer sans aucune course de données.

Architecture imposée

Package	Rôle
`lexer`	Découpe le texte en tokens, avec leur position
`dsl`	Analyse, vérifie et compile le langage de règles

Package	Rôle
`engine`	Exécute les règles compilées sur les tokens, en flux et en parallèle
`diag`	Construit et affiche les diagnostics (texte et JSON)
`sourcemap`	Construit la carte des sources
`lsp`	Serveur de langage (JSON-RPC 2.0 sur l'entrée et la sortie standard)
`naive`	Moteur de référence, simple et lent, utilisé uniquement par les tests (partie 8)
`main`	Sous-commandes, options, codes de sortie

Règle d'or : le package `engine` ne connaît **aucun** modificateur par son nom. Il ne connaît que les primitives du langage (partie 2.3). `up`, `hex`, `title` et tous les autres sont définis dans des fichiers de règles.

Partie 1 : Compatibilité avec go-reforged

Avec les règles standard (partie 2.5), tous les exemples du sujet de go-reforged doivent donner exactement le même texte de sortie, avec deux différences :

- le format des diagnostics change (partie 4) ;
- l'option `--articles` est supprimée : les exceptions d'articles s'écrivent désormais avec des règles `rewrite` (partie 2.2).

La ponctuation, les guillemets, les parenthèses et la majuscule de début de phrase restent gérés directement par le moteur. Tout le reste passe par le langage de règles.

Partie 2 : Le langage de règles

2.1 Grammaire

Les fichiers de règles portent l'extension `.gr` et sont encodés en UTF-8. Voici leur grammaire, en notation EBNF :

```
file      = { import | decl } ;
import    = "import" STRING ;
decl      = [ "override" ] ( modifier | block | rewrite ) ;
modifier  = "modifier" IDENT "(" [ param ] ")" "=" selector { "|>" call } ;
param     = IDENT ":" "int" [ "=" INT ] ;
selector  = "words" "(" ( IDENT | INT ) ")" ;
block     = "block" IDENT "=" call { "|>" call } ;
rewrite   = "rewrite" IDENT ":" REGEX "before" REGEX "->" STRING [ "keep_case" ] ;
call      = IDENT [ "(" [ arg { "," arg } ] ")" ] ;
arg       = INT | STRING | list ;
list      = "[" [ STRING { "," STRING } ] "]" ;

IDENT     = letter { letter | digit | "-" | "_" } ;
INT       = [ "-" ] digit { digit } ;
STRING    = '"' { any character except '"' and newline, or '\"' } '"' ;
REGEX     = "/" { any character except "/" and newline, or "\/" } "/" [ "i" ] ;
comment   = "#" { any character } newline ;
```

Les espaces, les tabulations, les sauts de ligne et les commentaires peuvent apparaître entre deux éléments et sont ignorés.

2.2 Les trois sortes de déclarations

`modifier` déclare un modificateur postfixé, utilisable dans le texte sous la forme `(nom)` ou `(nom, n)`. Le sélecteur `words(n)` choisit les `n` mots précédents, puis chaque fonction du pipeline `|>` est appliquée à chacun de ces mots, de gauche à droite. Si le modificateur n'a pas de paramètre, `(nom, n)` est un modificateur invalide.

`block` déclare un bloc, utilisable dans le texte sous la forme `[nom]...[/nom]`. Le pipeline est appliqué à chaque mot contenu dans le bloc.

`rewrite` déclare une règle de réécriture : si un mot correspond à la première expression régulière **et** que le mot suivant correspond à la seconde, le mot est remplacé par la chaîne donnée. Avec `keep_case`, le remplacement adopte la casse du mot d'origine : tout en minuscules, majuscule initiale, ou tout en majuscules. Un mot d'une seule lettre majuscule compte comme « majuscule initiale » (`A` -> `An`). Les règles `rewrite` remplacent l'étape « Articles » de go-reforged.

Ordre de priorité :

- Pour un même mot, les règles `rewrite` sont essayées de la **dernière déclarée à la première**. La première qui correspond est appliquée, les autres sont ignorées.
- Les fichiers sont chargés dans cet ordre : `std.gr`, puis les fichiers donnés avec `--rules`, dans l'ordre de la ligne de commande. Un fichier importé est chargé à l'endroit de son `import`.
- Déclarer un `modifier` ou un `block` qui existe déjà est une erreur, sauf si la déclaration commence par `override`.

2.3 Primitives

Le langage a deux types : `text` et `int`. Un mot sélectionné est toujours un `text`, et le résultat final d'un pipeline doit être un `text`.

Fonction	Type	Effet
`upper`	text → text	Met en majuscules (Unicode)
`lower`	text → text	Met en minuscules (Unicode)
`capitalize`	text → text	Majuscule à la première lettre, minuscules ensuite
`reverse`	text → text	Inverse les caractères (runes)
`title(small)`	text → text	Comme `capitalize`, sauf pour les mots de la liste `small` qui passent en minuscules ; le premier mot de la sélection prend toujours une majuscule
`parse_int(base)`	text → int	Lit un entier en base 2 à 36 ; échec si invalide ou hors `int64`

Fonction	Type	Effet
`format_int(base)`	int → text	Écrit un entier en base 2 à 36, chiffres en majuscules
`parse_roman`	text → int	Lit un chiffre romain valide
`format_roman`	int → text	Écrit un nombre entre 1 et 3999 en chiffres romains

Si une primitive échoue sur un mot, **aucun** mot de la sélection n'est modifié, le modificateur reste tel quel dans le texte et un avertissement est produit, comme dans go-reforged.

2.4 Vérification

Avant de traiter le moindre texte, le programme charge et vérifie **tous** les fichiers de règles. Les erreurs suivantes doivent être détectées, avec leur position exacte dans le fichier `.gr` :

- erreur de syntaxe ;
- fonction inconnue, ou mauvais nombre ou mauvais type d'arguments ;
- erreur de type dans un pipeline (ex : `upper` reçoit un `int`) ;
- pipeline dont le résultat n'est pas un `text` ;
- expression régulière invalide ;
- base hors de l'intervalle 2–36 ;
- déclaration en double sans `override`, ou `override` d'une déclaration qui n'existe pas ;
- `import` circulaire, ou `import` qui sort du dossier autorisé (partie 6).

Si un fichier de règles contient une erreur, **aucun** texte n'est traité.

2.5 La bibliothèque standard

Le fichier `std.gr` définit tous les modificateurs, blocs et règles d'articles de go-reforged. Il doit être **intégré au binaire** avec le package `embed`, et l'option `--no-std` permet de ne pas le charger. En voici le début ; à toi d'écrire le reste :

```
# std.gr — bibliothèque standard de go-ascended

modifier up(n: int = 1)  = words(n) |> upper
modifier low(n: int = 1) = words(n) |> lower
modifier hex(n: int = 1) = words(n) |> parse_int(16) |> format_int(10)

block up = upper

rewrite a-an: /^a$/i before /^[aeiouh]/i -> "an" keep_case
```

Partie 3 : Constructions sur plusieurs lignes

Dans go-reforged, tout s'arrêtait au saut de ligne. Ici, l'unité de traitement devient le **paragraphe** : un ensemble de lignes séparé des autres par au moins une ligne vide.

- Les blocs et les guillemets peuvent s'étendre sur plusieurs lignes d'un même paragraphe. Les sauts de ligne qu'ils contiennent sont conservés.
- Un modificateur peut compter des mots situés sur les lignes précédentes du même paragraphe.
- Rien ne traverse jamais une ligne vide.
- Un paragraphe ne peut pas dépasser `--max-paragraph` octets (8 Mo par défaut). Au-delà, c'est une erreur de limite (partie 6.2).

Les octets qui ne forment pas de l'UTF-8 valide sont recopiés tels quels dans la sortie et produisent un avertissement. Ils ne font jamais partie d'un mot.

Partie 4 : Diagnostics et carte des sources

4.1 Format texte

Chaque diagnostic a un **code** stable : `GR1xx` pour le texte traité, `GR2xx` pour les fichiers de règles, `GR3xx` pour la sécurité et les limites. Le format texte affiche la ligne en cause et souligne l'élément concerné :

```
sample.txt:1:8: warning[GR101]: cannot parse "ZZ" as base-16 integer
  |
1 | Add ZZ (hex) and 3 (bin) please (up, -1) .
  |        ^^^^^
```

- Les colonnes sont comptées en caractères (runes) et commencent à 1.
- Une tabulation dans la ligne affichée compte pour une colonne, et le soulignement doit rester aligné.
- La liste complète des codes et de leur signification doit figurer dans un fichier `DIAGNOSTICS.md`.

4.2 Format JSON

Avec `--format json`, chaque diagnostic est écrit sur `stderr` sous la forme d'un objet JSON par ligne :

```
{"file":"sample.txt","line":1,"column":8,"endLine":1,"endColumn":13,"severity":"warning","cod
e":"GR101","message":"cannot parse \"ZZ\" as base-16 integer"}
```

`endColumn` est exclusive : l'élément occupe les colonnes 8 à 12.

4.3 Carte des sources

Avec `--source-map <file>`, le programme écrit un fichier JSON qui relie chaque mot de la sortie aux éléments de l'entrée qui l'ont produit (le mot d'origine **et** les modificateurs qui l'ont transformé). Les positions sont au format `[ligne, colonne, ligne de fin, colonne de fin exclusive]`.

```
$ cat sample.txt
Simply add 42 (hex) .
$ go run . run --source-map map.json sample.txt result.txt
$ cat result.txt
Simply add 66.
```

Dans `map.json`, le mot `66` doit apparaître ainsi :

```
{"out":[1,12,1,14],"in":[1,12,1,20]}
```

La carte des sources doit rester exacte avec `--workers` supérieur à 1 et pour les constructions sur plusieurs lignes.

Partie 5 : Serveur de langage (LSP)

La sous-commande `lsp` lance un serveur qui respecte le Language Server Protocol. Il communique en JSON-RPC 2.0 sur l'entrée et la sortie standard, chaque message étant précédé d'un en-tête `Content-Length`. Tu dois l'implémenter toi-même avec `encoding/json` et `bufio`.

5.1 Méthodes obligatoires

Méthode	Comportement attendu
`initialize` / `initialized`	Annonce les capacités du serveur
`textDocument/didOpen`	Mémorise le document et publie ses diagnostics
`textDocument/didChange`	Synchronisation complète du document, puis nouveaux diagnostics
`textDocument/didClose`	Oublie le document et efface ses diagnostics
`textDocument/publishDiagnostics`	Notification envoyée par le serveur, avec les mêmes diagnostics que la ligne de commande
`textDocument/hover`	Sur un modificateur ou une balise : affiche sa déclaration `.gr` et le fichier d'origine
`textDocument/codeAction`	Sur un diagnostic `GR101`, `GR102` ou `GR103` : propose de supprimer le modificateur invalide
`textDocument/formatting`	Renvoie le texte entièrement transformé
`shutdown` / `exit`	Arrêt propre ; code de sortie `0` si `shutdown` a été reçu avant `exit`, sinon `1`

Les fichiers `.gr` ouverts dans l'éditeur doivent eux aussi recevoir leurs diagnostics (erreurs de syntaxe et de type).

5.2 Positions UTF-16

Le protocole compte les positions en **unités de code UTF-16**, à partir de **0**, alors que ta ligne de commande compte en runes à partir de 1. Pour la ligne `😀😀 ZZ (hex)`, le même diagnostic donne :

- en ligne de commande : `1:7` ;
- en LSP : `"start": {"line": 0, "character": 8}`, `"end": {"line": 0, "character": 13}`.

5.3 Robustesse

- Un message mal formé (JSON invalide, `Content-Length` faux ou absent, méthode inconnue) produit la bonne erreur JSON-RPC, sans jamais arrêter le serveur.
- Un en-tête `Content-Length` supérieur à 64 Mo est refusé.

- Les diagnostics d'un document volumineux sont calculés sans bloquer le traitement des autres messages. Si le document change pendant le calcul, l'ancien résultat est abandonné et n'est jamais publié.

Partie 6 : Sécurité

Considère que le texte, les fichiers de règles et les messages LSP peuvent venir d'un **attaquant**.

6.1 Imports

- Un `import` est résolu par rapport au dossier du fichier qui le contient.
- Aucun `import` ne peut sortir du dossier du premier fichier `--rules` (la « racine des règles »). Cela concerne les chemins absolus, les `..` et les liens symboliques qui pointent hors de la racine. Utilise `os.Root`.
- Les imports circulaires sont détectés et signalés avec la chaîne complète (`a.gr -> b.gr -> a.gr`).
- `std.gr` ne peut pas être importé : il est toujours chargé en premier, sauf avec `--no-std`.

6.2 Limites de ressources

Option	Valeur par défaut	Ce qu'elle limite
`--max-depth`	64	Profondeur d'imbrication des blocs et des guillemets
`--max-count`	10000	Nombre `n` dans un modificateur `(nom, n)`
`--max-paragraph`	8 Mo	Taille d'un paragraphe
`--max-imports`	32	Nombre total de fichiers de règles chargés
`--max-rule-size`	1 Mo	Taille d'un fichier de règles

Le dépassement d'une limite est une **erreur** `GR3xx` avec le code de sortie `4`. Il ne doit jamais provoquer de plantage, de blocage ou une consommation de mémoire excessive.

Les expressions régulières utilisent obligatoirement le package `regexp` de Go. Explique dans `SECURITY.md` pourquoi ce choix protège contre les attaques ReDoS.

6.3 Écriture du fichier de sortie

- Le fichier temporaire est créé avec `os.CreateTemp` dans le dossier de destination, avec les droits `0600`.
- Si le fichier de sortie existe déjà, ses droits sont conservés.
- Si le fichier de sortie est un lien symbolique, le programme refuse d'écrire (erreur `GR3xx`), sauf avec l'option `--follow-symlinks`.
- Le fichier temporaire est supprimé en cas d'erreur, y compris après un `Ctrl+C` (`os/signal`).

6.4 Modèle de menaces

Fournis un fichier `SECURITY.md` qui décrit :

- ce qu'un attaquant contrôle (texte, règles, messages LSP, système de fichiers) ;
- ce qu'il pourrait chercher à obtenir (lecture de fichiers, déni de service, écrasement de fichiers, plantage) ;
- pour chaque menace, la protection mise en place et le test qui la vérifie.

6.5 Ce que les auditeurs essaieront

Voici une partie de ce que les auditeurs vont tester. La liste n'est pas exhaustive.

- 100 000 blocs imbriqués : `[up][up][up]...`
- `(up, 9999999999999999999)`
- un fichier de règles contenant `import "../../../etc/passwd"`
- un lien symbolique dans la racine des règles qui pointe vers `/etc`
- un fichier de sortie qui est un lien symbolique vers un fichier important
- un paragraphe de 2 Go sans ligne vide
- du binaire aléatoire en entrée
- un message LSP avec `Content-Length: 99999999999`
- deux fichiers de règles qui s'importent mutuellement

Partie 7 : Performance

- Le programme traite des fichiers de **5 Go**. Avec les options par défaut et `--workers 4`, la mémoire utilisée (RSS maximale) reste sous **256 Mo**, quelle que soit la taille du fichier.
- Les paragraphes sont traités en parallèle. Le résultat, les diagnostics et la carte des sources sont **identiques octet par octet** quel que soit `--workers`.
- Les options `--cpuprofile <file>` et `--memprofile <file>` produisent des profils lisibles avec `go tool pprof` (package `runtime/pprof`).
- Fournis un fichier `PERFORMANCE.md` avec tes mesures : débit en Mo/s pour 1, 2, 4 et 8 workers, mémoire maximale, et une capture d'un profil CPU commentée qui montre une optimisation que tu as faite.

Partie 8 : Tests

- Couverture d'au moins **85 %** sur l'ensemble du projet, et d'au moins **95 %** sur le package `dsl`.
- **Tests dorés** : un dossier `testdata/golden/` contient des triplets `nom.in`, `nom.out` et `nom.diag`. L'option `go test ./... -update` régénère les fichiers attendus.
- **Fuzzing** : au minimum `FuzzProcess` (texte), `FuzzParseRules` (fichiers `.gr`) et `FuzzLSPFraming` (flux de messages LSP).

- **Test différentiel** : le package `naive` contient une implémentation volontairement simple, qui charge tout en mémoire et travaille sur un seul thread. Un test de fuzzing vérifie que `naive` et `engine` produisent toujours la même sortie et les mêmes diagnostics.
- **Formateur de règles** : la sous-commande `fmt-rules` réécrit un fichier `.gr` dans un style canonique. Un test de fuzzing vérifie qu'elle est **idempotente** et qu'elle ne change pas le sens des règles.

Partie 9 : Ligne de commande

```
$ go run . <input> <output>
$ go run . run [options] <input> <output>
$ go run . check [options] <input>
$ go run . lsp [options]
$ go run . explain <name>
$ go run . fmt-rules <file.gr>
```

- `go run . <input> <output>` reste un raccourci pour `run`, pour rester compatible avec go-reloaded.
- `explain <name>` affiche la déclaration d'un modificateur ou d'un bloc et le fichier où elle se trouve.
- Les options de go-reforged sont conservées (sauf `--articles`), en plus de : `--rules <file>` (répétable), `--no-std`, `--format text|json`, `--source-map <file>`, `--follow-symlinks`, les limites de la partie 6.2 et les options de profilage de la partie 7.

Code	Signification
`0`	Succès (des avertissements ont pu être affichés)
`1`	Erreur d'utilisation ou de fichier
`2`	Erreur de contenu ou erreur dans un fichier de règles
`3`	En mode `check` : des avertissements ont été trouvés
`4`	Limite de ressources dépassée ou violation de sécurité

Utilisation

```
$ cat rules/main.gr
import "articles.gr"

modifier shout(n: int = 1) = words(n) |> upper |> reverse
override modifier cap(n: int = 1) = words(n) |> title(["of", "the"])
$ cat rules/articles.gr
rewrite hour: /^an?$/i before /^(hour|honest|heir)/i -> "an" keep_case
rewrite house: /^an?$/i before /^(house|university|unicorn)/i -> "a" keep_case
$ cat sample.txt
Bring a umbrella to a university in a hour , said the lord of the rings (cap, 5) .
stop (shout)
$ go run . run --rules rules/main.gr sample.txt result.txt
$ cat result.txt
Bring an umbrella to a university in an hour, said The Lord of the Rings.
POTS
$ cat bad.gr
modifier broken(n: int = 1) = words(n) |> parse_int(16) |> upper
$ go run . run --rules bad.gr sample.txt result.txt
bad.gr:1:60: error[GR201]: type mismatch: "upper" expects text, got int
```

```
  |
1 | modifier broken(n: int = 1) = words(n) |> parse_int(16) |> upper
  |                                                            ^^^^^
$ echo $?
2
$ cat evil.gr
import "../../secret.gr"
$ go run . run --rules evil.gr sample.txt result.txt
evil.gr:1:8: error[GR301]: import escapes rules root: "../../secret.gr"
  |
1 | import "../../secret.gr"
  |        ^^^^^^^^^^^^^^^^^
$ echo $?
4
$ cat multi.txt
He said : " this is
a [up]long
quote[/up] " .
$ go run . multi.txt result.txt
$ cat result.txt
He said: "this is
a LONG
QUOTE".
$ go run . explain hex
modifier hex(n: int = 1) = words(n) |> parse_int(16) |> format_int(10)
  defined in <std>/std.gr:5
```

Bonus

- Prise en charge de `$/cancelRequest` dans le serveur LSP.
- Sous-commande `repl` : un terminal interactif où l'on tape une ligne de texte et où l'on voit immédiatement le résultat et les diagnostics.
- Primitive `when(REGEX, call)` qui n'applique une fonction qu'aux mots qui correspondent à l'expression régulière, avec sa vérification de types.
- Extension VS Code minimale (un simple fichier `package.json` et quelques lignes de JavaScript) qui lance `go run . lsp`.

Ce projet t'aidera à découvrir

- La conception d'un langage : grammaire, analyse syntaxique, vérification de types et messages d'erreur
- Le Language Server Protocol, JSON-RPC 2.0 et le codage UTF-16
- La sécurité des entrées non fiables : parcours de répertoires, liens symboliques, déni de service, ReDoS
- La rédaction d'un modèle de menaces
- Le traitement en flux de très gros fichiers avec un état qui s'étend sur plusieurs lignes
- La concurrence déterministe, l'annulation de calculs et le détecteur de courses de données
- Le profilage avec `pprof` et l'optimisation mesurée
- Les tests dorés, le fuzzing et le test différentiel